

Specification of
DevAtServ
v. 0.14.4.16

Implemented by:

Thomas Pollerspöck

Nguyen Huynh Tri Cuong

Tran Duy Ngoan

Mai Dinh Nam Son

Hua Van Thong

Holger Queckenstedt

03/2026

Contents

1	Introduction	1
2	Installation and start	2
3	Components	5
4	Environment	8
5	Code analysis	10
6	Logging	12
6.1	Background	12
6.2	Summary	12
7	Library documentation	13
7.1	MicroserviceBase	14
7.2	MicroserviceClewareSwitch	29
8	Appendix	57
8.1	Installed Python Modules	57

Chapter 1

Introduction

The Robot Framework is an automation framework that can be used to execute automated software tests. It's open source, keyword driven, and extensible. Tests are implemented in simple text files.

The extensibility of the Robot Framework gives every test developer the possibility to add missing functionality easily. But when thinking about missing functionality, some aspects urgently need to be considered:

- Additions should be designed carefully to make them as much generic as possible. This will make the additions useful for a wide range of test developers.
- Additions have to be bundled in a meaningful way, and they have to be set under version control.
- Tests need to be reproduced, and therefore also the entire test system (Robot Framework together with all additions) must be reproducible.

And these deliberations are not only related to the Robot Framework itself. Developing software in an efficient way also requires a development environment. And this development environment most probably needs to be configured - at least to be able to handle the additions also. It has to be ensured to use open source software only, to avoid any license fee. And finally it must be ensured that every test developer within a team of developers uses the same development environment with the same settings.

To cover all aspects mentioned above, the following things have been done:

- Download a certain Python version (because the Robot Framework is a Python based application).
- Add further useful Python modules to the Python installation.
- Download a certain version of the Robot Framework. This is the base.
- Implement (carefully) some changes within the core source code of the Robot Framework.
- Add further components to provide useful features that are currently missing in Robot Framework (like a management of test configuration values and a version control mechanism).
- Select a development environment (here: VSCodium).
- Configure this development environment to handle especially the Robot Framework code together with all additions.
- Put all things together in a separate installer.

The outcome is one single installer, that installs all together: the framework, additional libraries, the development environment and all settings.

This is what we call "**RobotFramework AIO**", with **AIO** means: **All In One**.

With this solution the entire test setup is reproducible by every developer on every computer.

And there is no need any more for any user, to install all affected components manually, and there is also no need any more for any user, to spend effort on the configuration of the test setup. After using the RobotFramework AIO installer, test developers immediately can start to implement tests.

Chapter 2

Installation and start

The **RobotFramework AIO** can be installed under Windows and under Linux in this way:

- **Win10**

Double-click the `setup_*.exe` file and follow the instructions.

- **Linux**

If already installed, remove the previous version (this will not remove your own data):

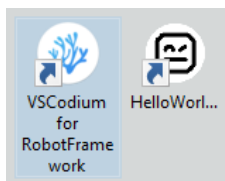
```
sudo apt-get remove robotframework-aio
```

Install/Update now the current version (assumed you are in the download directory):

```
sudo apt-get install ./setup_RobotFramework_AIO_OSD6Linux\/*.deb
```

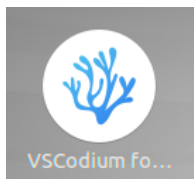
After the installation is finished, a new **VSCodium** icon is present on desktop:

- **Win10**



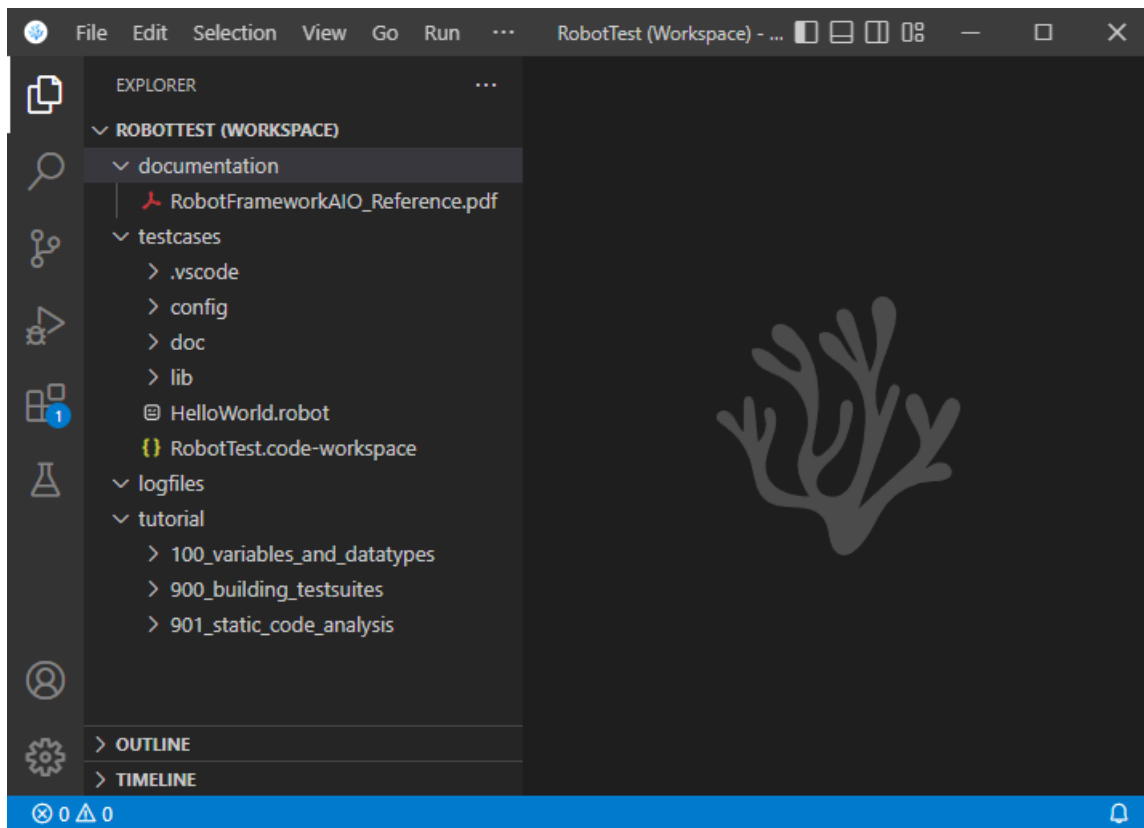
(together with a link to the example robot file `HelloWorld.robot`)

- **Linux**



A double-click starts the application.

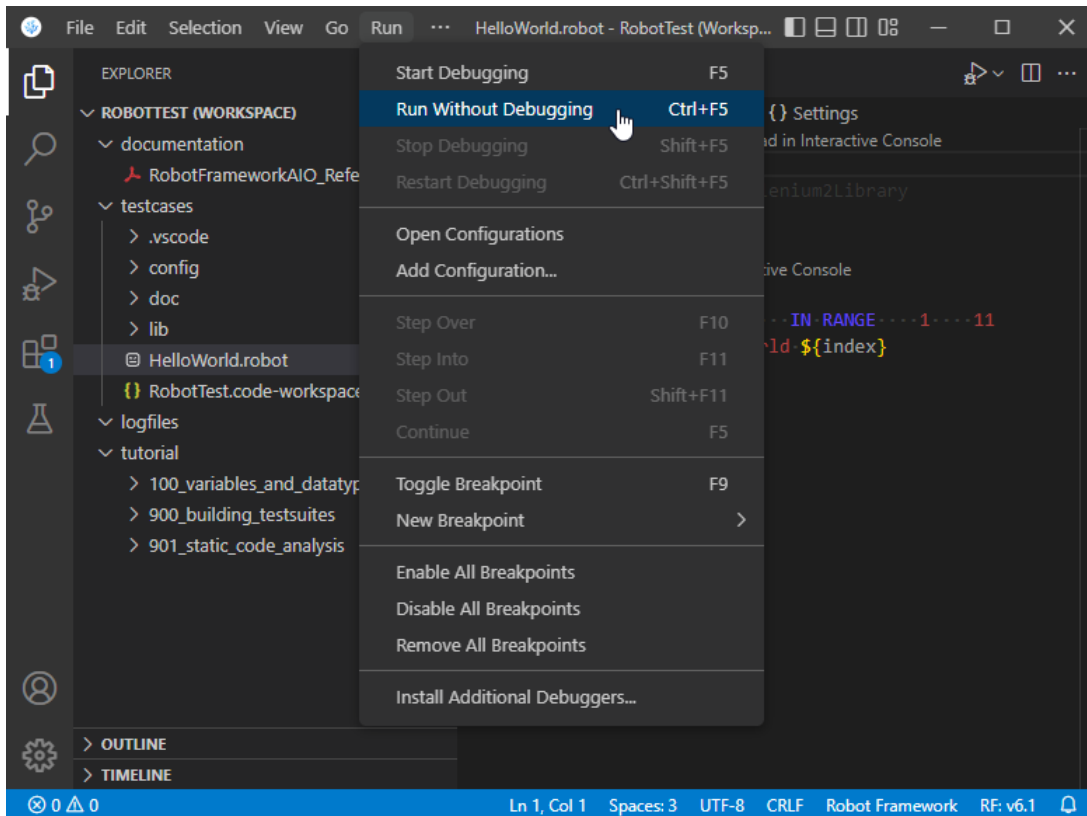
On first start the **VSCodium** main window appears like this:



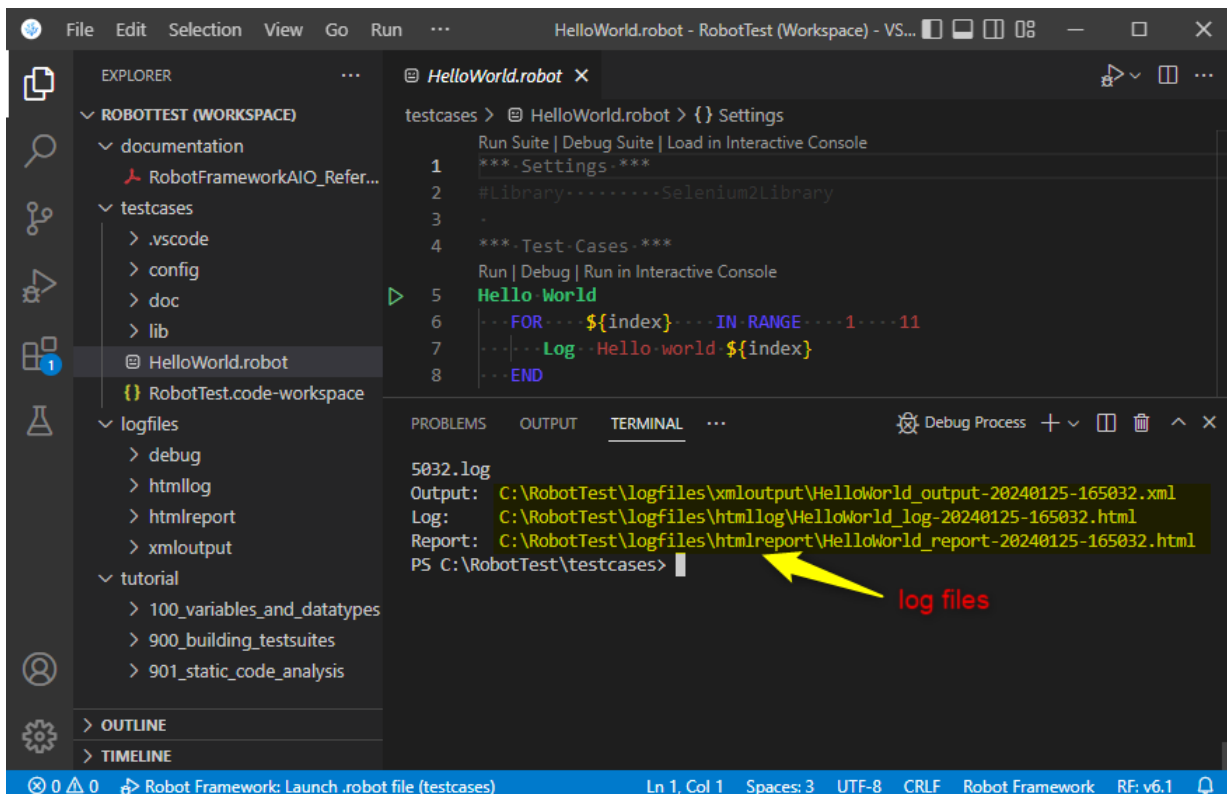
The preconfigured workspace contains the following parts:

- The main documentation `RobotFrameworkAIO_Reference.pdf` (including the documentation of all components that are part of the **RobotFramework AIO**)
- A `testcases` folder containing (beneath other elements) the example robot file `HelloWorld.robot`
- A `logfiles` folder in which all log files and reports will be placed (from tests that are executed with **VSCodium**)
- A `tutorial` folder containing a large bunch of testfiles that can be used for self-dependent learning

For the usage of the predefined infrastructure (e.g. the `logfiles` folder), the **VSCodium** provides an adapted configuration of the **Run** button. This button executes the robot file selected in project explorer of **VSCodium**.



After test execution, the links to log files and reports can be found in the **TERMINAL** window.



Every file there can be opened by CTRL+click on the selected file.

Chapter 3

Components

Compared with the core Robot Framework, the RobotFramework AIO contains some additional components, that extend the functionality by useful features. All components are hosted on GitHub. Most of the components can also be used stand-alone (= outside the context of RobotFramework AIO). But in case of a stand-alone usage, the user is responsible for the installation and configuration. Further hints can be found in the readme files of the corresponding GitHub repository.

The following is an overview about all additional components, that are part of the RobotFramework AIO:

1. RobotFramework_TestsuitesManagement

The **RobotFramework_TestsuitesManagement** enables users to define dynamic configuration values within separate configuration files in JSON format. These configuration values are available during test execution - but under certain conditions that can be defined by the user (e.g. to realize a variant handling or to define parameters that are test bench specific).

The **RobotFramework_TestsuitesManagement** also provides a version control mechanism to ensure that the developed tests fit to the test environment.

Homepage: [RobotFramework_TestsuitesManagement](#)

2. JsonPreprocessor

The **JsonPreprocessor** provides additional features in JSON files. These features extend the standard JSON format and make it easier to use JSON files for e.g. configuring tests (see also **RobotFramework_TestsuitesManagement**). The additional features are:

- Add comments
- Let a JSON file import other JSON files (nested imports; e.g. to avoid redundancy)
- Allow also pure Python keywords like `True` , `False` and `None`
- Define new parameters and overwrite already existing ones

Homepage: [JsonPreprocessor](#)

3. QConnectBase

QConnectBase is a connection testing library for the Robot Framework. It provides a mechanism to handle trace log continuously receiving from a connection (such as Raw TCP, SSH, Serial, etc.) besides sending data back to the other side. It's especially efficient for monitoring the overflowed response trace log from asynchronous trace systems.

Homepage: [QConnectBase](#)

4. RobotLog2RQM

RobotLog2RQM writes Robot Framework test results (available in XML format) to the IBM® Rational® Quality Manager (RQM). This includes:

- Create all required resources (Test Case Execution Record, Test Case Execution Result, ...) for new testcase on RQM
- Link all testcases to provided testplan

- Add new test result for existing testcase on RQM
- Update existing testcase on RQM

Homepage: [RobotLog2RQM](#)

5. RobotLog2DB

RobotLog2DB writes Robot Framework test results (available in XML format) to a database. This database is provided by another component called **TestResultWebApp**. Based on the content of the database the **TestResultWebApp** presents an overview about the whole test execution and details about each test result.

Homepage: [RobotLog2DB](#)

6. PyTestLog2DB

PyTestLog2DB writes test results of the Python module pytest (available in XML format) to a database. This database is provided by another component called **TestResultWebApp**. Based on the content of the database the **TestResultWebApp** presents an overview about the whole test execution and details about each test result.

Homepage: [PyTestLog2DB](#)

7. TestResultWebApp

TestResultWebApp is a web-based application and is used for visualizing and tracking test execution results. It provides charts from an overview of the test result to the detail of all included test cases. It also provides tools to control the quality of developing software version (under testing) by the a graphical comparison of test results from different test executions

Homepage: [TestResultWebApp](#)

8. GenPackageDoc

GenPackageDoc provides a toolchain to generate a documentation in PDF format out of Python sources that are stored within a repository. The content of this documentation is taken out of the docstrings of functions, classes and their methods. All docstrings have to be written in reStructuredText (RST) format, that is a certain markdown dialect. It is possible to extend the documentation by the content of additional files either in RST format or in LaTeX format. **GenPackageDoc** is also designed to consider setup informations of a repository.

Homepage: [GenPackageDoc](#)

9. PythonExtensionsCollection

The **PythonExtensionsCollection** extends the functionality of Python by some useful functions that are not available in Python immediately. This covers for example file and folder operations, string operations like normalizing a path and a pretty print method. Also a file comparison feature is available.

Homepage: [PythonExtensionsCollection](#)

10. RobotframeworkExtensions

The **RobotframeworkExtensions** extend the functionality of the Robot Framework by some useful keywords. This covers for example string operations like normalizing a path and a pretty print keyword (especially for composite Python data types). The **RobotframeworkExtensions** keywords are implemented in Python (see **PythonExtensionsCollection**).

Homepage: [RobotframeworkExtensions](#)

11. Tutorial

The RobotFramework AIO tutorial contains robot code examples together with the documentation explaining how to use these examples and how they work. The basic idea behind this tutorial is not to have theory only but additionally to this theory also the possibility to experience what is explained inside. The main focus of this tutorial is on some feature extensions available for the Robot Framework.

Homepage: [Tutorial](#)

12. Development environment

The RobotFramework AIO installer also installs VSCodium, that is an open source editor. VSCodium is preconfigured especially for the usage together with the RobotFramework AIO.

Features:

- Workspace with included testcases folder, documentation and tutorial
- Previews for certain formats like, MD, RST, PDF, PlantUML
- Support of the extended JSON syntax of **RobotFramework_TestsuitesManagement**
- Static code analysis by Robocop, that is an also included static code checker

Chapter 4

Environment

During an installation of the RobotFramework AIO all content is placed at two different main locations:

- `RobotFramework`
Contains the framework installation files itself
- `RobotTest`
Contains the *playground* for the user, including default folder for testcases and logfiles

The root path to these folders can be set by the user during the installation process (called `<root>` in listings below).

The following overview contains a list of environment variables that are introduced by the installer. They can be used to refer to locations inside these folders.

- `RobotLogPath`
Log files folder in which the RobotFramework AIO places the log files per default.
Location: `<root>/RobotTest/logfiles`
- `RobotTestPath`
Folder in which the user can place the testcases.
Location: `<root>/RobotTest/testcases`
- `RobotTutorialPath`
Folder containing a tutorial of the RobotFramework AIO (useful for self learning).
Location: `<root>/RobotTest/tutorial`
- `RobotPythonPath`
Folder containing the Python installation.
Location: `<root>/RobotFramework/python39`
- `RobotScriptPath`
Folder containing scripts of the Python installation.
Location: `<root>/RobotFramework/python39/scripts`
- `RobotVsCode`
Folder containing the installation of *Visual Studio Code* that is intended to be the main development environment for the RobotFramework AIO.
Location: `<root>/RobotFramework/robotvscode`
- `RobotToolsPath`
Tools which require and extend functionality of the Robot Framework.
Location: `<root>/RobotFramework/tools`
- `RobotDevtools`
Tools which are independent of Robot Framework, but are bundled and tested with RobotFramework AIO.
Location: `<root>/RobotFramework/devtools`

- **RobotAppium**

Location of Appium tested and distributed with RobotFramework AIO. More information about Appium you find here: [appium](#)

Location: `<root>/RobotFramework/devtools/Windows/Appium`

- **RobotNodeJS**

Root folder of `nodjs` tested and distributed with RobotFramework AIO. More information about `nodjs` you you find here: [nodejs.org](#)

Location: `<root>/RobotFramework/devtools/Windows/nodejs`

Chapter 5

Code analysis

The RobotFramework AIO installation provides a static code analyser to detect potential errors and violations to coding conventions. The name of the analyser is **Robocop**.

Robocop is integrated in Visual Studio Code and is triggered automatically in case of

1. a file is opened in editor,
2. a file is changed and saved.

The outcome will look like this:

```

suite02.robot 3 X
robotframework-tutorial > 901_static_code_analysis > suite02.robot > ...
31 Test Case 0201
32 ...[Documentation]...Documentation of test case
33 ...Log...I am test '${TEST_NAME}' of suite '${SUITE_NAME}'...console=yes
34 ...#TODO::Add return value to keyword implementation; check return value here
35 ...test_keyword_1
36
37 Test Case 0201
38 ...[Documentation]...Documentation of test case
39 ...Log...I am test '${TEST_NAME}' of suite '${SUITE_NAME}'...console=yes
40
41
42
43

```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL Filter (e.g. text, **/*.ts, !**/node_modules/**)

- Multiple test cases with name 'Test Case 0201' (first occurrence in line 31) robocop(0801) [37, 1]
- Found TODO in comment robocop(0701) [34, 7]
- Too many blank lines at the end of file robocop(1010) [43, 1]

Further examples can be found in the RobotFramework AIO tutorial, chapter [901_static_code_analysis](#).

How to configure?

Robocop contains a set of rules that are used to check the source files of the RobotFramework AIO. Not all of them are really useful and some of them are also against our company internal development rules. Therefore we have the need to exclude rules from Robocop execution.

Some rules can be configured (e.g. the rule checking the maximum number of characters within a line of code). Also here we have the need for careful adaptations.

When Robocop is triggered automatically within Visual Studio Code, then the configuration is done with the help of a `.robocop` file. This file has to be placed within the projects folder (see tutorial). To make Robocop results comparable this configuration file should not be modified - or at least should be kept aligned with the version all other project team members use.

Command line

In addition to the automated triggering within Visual Studio Code Robocop can also be called in command line. This is useful

1. in case of a complete folder has to be checked in one step (to avoid the need to open every single source file inside separately),
2. in case of an alternative configuration is wanted (temporarily; to avoid to manipulate the standard configuration file),
3. in case of Robocop shall be triggered by automats like Jenkins,
4. in case of the Robocop results are needed within a log file (output to a log file needs to be configured separately).

To give it a try make a copy of the configuration file `.robocop` available within the tutorial, and give this copy any other name (e.g. `robocop.arg`).

Now Robocop can be called with command lines like this:

1.) *Simple check of single file with default configuration:*

```
"%RobotPythonPath%/python.exe" -m robocop "<path>/mytestfile.robot"
```

2.) *Check of single file with individual configuration taken from argument file:*

```
"%RobotPythonPath%/python.exe" -m robocop --argumentfile "<path>/robocop.arg" ↔  
↔ "<path>/mytestfile.robot"
```

3.) *Check of entire folder with individual configuration taken from argument file:*

```
"%RobotPythonPath%/python.exe" -m robocop --argumentfile "<path>/robocop.arg" "<folder>"
```

How to activate?

In Visual Studio Code the automated triggering of Robocop is activated per default within

```
RobotFramework\robotvscode\data\user-data\User\settings.json
```

with the following switches set to `true` :

```
"robot.lint.enabled": true,  
"robot.lint.robocop.enabled": true,
```

Chapter 6

Logging

6.1 Background

The Robot Framework provides several log levels (also known as trace levels) to give users the possibility to control the amount of output in log files. The build in keyword `log` writes content to the log files. The optional parameter `level` of this keyword defines under which conditions (under which log level) the `log` keyword shall write the log message.

With command line parameter `--loglevel` a user defines this condition. If the log level of an executed `log` keyword matches to the log level defined for test execution, the log message is logged, otherwise not.

Available log levels in Robot Framework are: `ERROR`, `WARN`, `INFO`, `DEBUG` and `TRACE`. The default log level in Robot Framework is `INFO`. This level becomes active, if nothing else is specified by a user.

The log level `INFO` is also used by the Robot Framework itself, e.g. for the logging of informations about executed keywords. The impact is that every log file contains both the users output and the Robot Framework output. For larger tests this might cause unfavorable large log files.

Therefore the RobotFramework AIO provides an additional log level `USER`. Tests that are executed under this log level, do not contain the Robot Framework `INFO` messages any more (but for sure still contain errors and warnings).

Example code:

```
log    my log message    USER
```

Example execution:

```
python -m robot --loglevel USER -b logfile.log ./testfile.robot
```

6.2 Summary

In opposite to the Robot Framework (core), the RobotFramework AIO provides the following log levels to control the amount of logged content:

		log level in <code>log</code> keyword					
		ERROR	WARN	USER	INFO	DEBUG	TRACE
log level in command line <code>--loglevel</code>	ERROR	logged	-----	-----	-----	-----	-----
	WARN	logged	logged	-----	-----	-----	-----
	USER	logged	logged	logged	-----	-----	-----
	INFO	logged	logged	logged	logged	-----	-----
	DEBUG	logged	logged	logged	logged	logged	-----
	TRACE	logged	logged	logged	logged	logged	logged

The log level `USER` has been added to give users the possibility to log own content without all `INFO` messages logged by the Robot Framework (core).

Chapter 7

Library documentation

The following sections contain the documentation of additional libraries that are part of the DevAtServ.

DevAtServ bundle

Version 0.14.4.16 (from 03/2026)

Underlying Robot Framework (core)

Included libraries

MicroserviceBase
Version 0.1.1 (from 02.02.2023)
https://github.com/test-fullautomation/python-microservice-base
<i>TODO</i>

MicroserviceClewareSwitch
Version 0.1.2 (from 02.02.2023)
https://github.com/test-fullautomation/python-microservice-cleware-switch
<i>TODO</i>

7.1 MicroserviceBase

MicroserviceBase

v. 0.1.1

TODO

02.02.2023

Contents

1	Introduction	1
2	Description	2
2.1	ToDo	2
2.1.1	ToDo ToDo	2
3	ServiceBase.py	3
3.1	Class: ResultType	3
3.2	Class: ResponseMessage	3
3.2.1	Method: get_json	3
3.2.2	Method: get_data	3
3.3	Class: ServiceBase	3
3.3.1	Method: parse_arguments	3
3.3.2	Method: parse_spec_arguments	4
3.3.3	Method: close	4
3.3.4	Method: create_request_data	4
3.3.5	Method: connect_broker	4
3.3.6	Method: serve	4
3.3.7	Method: register_service	5
3.3.8	Method: unregister_service	5
3.3.9	Method: request_service	5
3.3.10	Method: get_svc_api_methods_dict	5
3.3.11	Method: get_svc_api_methods_info_dict	5
3.3.12	Method: parse_docstring	6
3.3.13	Method: svc_api_get_version	6
3.3.14	Method: svc_api_get_gui_files	6
3.3.15	Method: is_specific_request	6
3.3.16	Method: on_specific_request	6
3.3.17	Method: on_request	7
3.4	Docutils System Messages	7
4	ServiceRegistry.py	8
4.1	Function: signal_handler	8
4.2	Class: ServiceRegistry	8
4.2.1	Method: receive_services_information	8
4.2.2	Method: handle_update	8
4.2.3	Method: notify_updates	9
4.2.4	Method: svc_api_get_services_info	9

CONTENTS	CONTENTS
4.2.5 Method: <code>svc_api_get_realtime_update_exchange</code>	9
4.2.6 Method: <code>svc_api_update_alias_conf</code>	9
4.2.7 Method: <code>svc_api_get_alias_conf</code>	9
4.2.8 Method: <code>is_specific_request</code>	10
4.2.9 Method: <code>on_specific_request</code>	10
5 Appendix	11
6 History	12

B

Chapter 1

Introduction

TODO: Add introduction of **MicroserviceBase**.

The sources of **MicroserviceBase** are available in [GitHub](#).

Informations about how to install the **MicroserviceBase** can be found in the [README](#).

Chapter 2

Description

2.1 ToDo

Description

2.1.1 ToDo ToDo

Description

ToDo ToDo ToDo

Description

Chapter 3

ServiceBase.py

3.1 Class: ResultType

Result Types.

3.2 Class: ResponseMessage

Response message class

3.2.1 Method: get_json

Convert the response message to JSON format.

Returns:

/ Type: str /

The response message in JSON format.

3.2.2 Method: get_data

Retrieve the result data as a string.

Returns:

/ Type: str /

The result data as a string.

3.3 Class: ServiceBase

3.3.1 Method: parse_arguments

Parse basic service arguments from the command line.

Arguments:

- cmd_args

/ Condition: required / Type: list /

Command-line arguments to be parsed.

Returns:

/ Type: dict /

A dictionary containing the parsed arguments.

3.3.2 Method: parse_spec_arguments

Parse specific arguments for each customized service.

This method should be overridden by subclasses to handle service-specific arguments.

Arguments:

- `cmd_args`
/ *Condition*: required / *Type*: list /
Command-line arguments to be parsed.

Returns:

/ *Type*: dict /
A dictionary containing the parsed specific arguments.

3.3.3 Method: close

Close the service connection.

Returns:

(no returns)

3.3.4 Method: create_request_data

Create request data for a given method name and arguments.

Arguments:

- `method_name`
/ *Condition*: required / *Type*: str /
The name of the method for which the request is being created.
- `args`
/ *Condition*: required / *Type*: list /
The list of arguments for the method.

Returns:

/ *Type*: dict /
A dictionary containing the method name and arguments.

3.3.5 Method: connect_broker

Establish a connection to the broker.

Arguments:

- `**kwargs`
/ *Condition*: optional / *Type*: dict /
Additional keyword arguments for broker connection parameters.

Returns:

(no returns)

3.3.6 Method: serve

Call to start service serving.

Returns:

(no returns)

3.3.7 Method: register_service

Register a service to the ServiceRegistry.

Returns:

(no returns)

3.3.8 Method: unregister_service

Unregister a service from the ServiceRegistry.

Returns:

(no returns)

3.3.9 Method: request_service

Send a service request to a specific exchange with a given routing key.

Arguments:

- request_data
/ Condition: required / Type: dict /
The data for the service request.
- exchange_name
/ Condition: required / Type: str /
The name of the exchange to send the request to.
- routing_key
/ Condition: required / Type: str /
The routing key for the request.

Returns:

(no returns)

3.3.10 Method: get_svc_api_methods_dict

Retrieve all service API methods provided by the service (methods starting with the prefix '**svc_api_**').

Returns:

/ Type: dict /
A dictionary containing the names and references of all service API methods.

3.3.11 Method: get_svc_api_methods_info_dict

Retrieve information for all service APIs from the docstrings of the methods.

Arguments:

- methods_dict
/ Condition: required / Type: dict /
A dictionary containing the names and references of all service API methods.

Returns:

/ Type: dict /
A dictionary containing the information of all service APIs extracted from their docstrings.

3.3.12 Method: parse_docstring

Parse function's docstring to get arguments and return information.

Arguments:

- `docstring`
/ *Condition*: required / *Type*: str /
Function's docstring.

Returns:

/ *Type*: str /
Payloads of the waiting signal if received.

3.3.13 Method: svc_api_get_version

Get the service version.

Returns:

/ *Type*: str /
Version of the service.

3.3.14 Method: svc_api_get_gui_files**3.3.15 Method: is_specific_request**

Check if the request is a specific request.

Arguments:

- `request`
/ *Condition*: required / *Type*: object /
The request object to be checked.

Returns:

/ *Type*: bool /
True if the request is a specific request, otherwise False.

3.3.16 Method: on_specific_request

Handle the event when a specific request is received.

Arguments:

- `ch`
/ *Condition*: required / *Type*: pika.channel.Channel /
The channel object from the pika library.
- `method`
/ *Condition*: required / *Type*: pika.spec.Basic.Deliver /
The method object containing delivery information from the pika library.
- `props`
/ *Condition*: required / *Type*: pika.spec.BasicProperties /
The properties of the message from the pika library.

- `body`
/ *Condition*: required / *Type*: dict /
The body of the message as a dictionary.

Returns:

(no returns)

3.3.17 Method: `on_request`

Handle an incoming request.

Arguments:

- `ch`
/ *Condition*: required / *Type*: `pika.channel.Channel` /
The channel object from the pika library.
- `method`
/ *Condition*: required / *Type*: `pika.spec.Basic.Deliver` /
The method object containing delivery information from the pika library.
- `props`
/ *Condition*: required / *Type*: `pika.spec.BasicProperties` /
The properties of the message from the pika library.
- `body`
/ *Condition*: required / *Type*: bytes /
The body of the message as bytes.

Returns:

(no returns)

3.4 Docutils System Messages**System Message**

ERROR/3 in <string>, line 186

Unknown target name: "svc_api".

Chapter 4

ServiceRegistry.py

4.1 Function: signal_handler

Handle signals from the operating system.

Arguments:

- sig
/ *Condition*: required / *Type*: int /
The signal number received from the OS.
- frame
/ *Condition*: required / *Type*: frame object /
The current stack frame.
- obj
/ *Condition*: required / *Type*: object /
The object that is handling the signal.

Returns:

(no returns)

4.2 Class: ServiceRegistry

Manage information for all services connected to the broker it is connected to.

This class handles the registration, updates, and retrieval of service information, ensuring that all services interacting with the broker are properly managed and monitored.

4.2.1 Method: receive_services_information

Run in a thread to listen for any changes from the services.

Returns:

(no returns)

4.2.2 Method: handle_update

Handle the event when service information is updated.

Arguments:

- ch
/ *Condition*: required / *Type*: pika.channel.Channel /
The channel object from the pika library.

- `method`
/ *Condition*: required / *Type*: `pika.spec.Basic.Deliver` /
The method object containing delivery information from the pika library.
- `properties`
/ *Condition*: required / *Type*: `pika.spec.BasicProperties` /
The properties of the message from the pika library.
- `body`
/ *Condition*: required / *Type*: `bytes` /
The body of the message as bytes.

Returns:*(no returns)***4.2.3 Method: `notify_updates`**

Notify updates to the realtime update channel.

Returns:*(no returns)***4.2.4 Method: `svc_api_get_services_info`**

Retrieve information of all services connected to the broker that the Service Registry is connected to.

Returns:

/ *Type*: `dict` /
A dictionary containing information of all connected services.

4.2.5 Method: `svc_api_get_realtime_update_exchange`

Retrieve the exchange name of the realtime update exchange.

Returns:

/ *Type*: `str` /
The name of the realtime update exchange.

4.2.6 Method: `svc_api_update_alias_conf`

Update the alias configuration information.

Arguments:

- `alias_string`
/ *Condition*: required / *Type*: `str` /
The alias configuration string to be updated.

Returns:*(no returns)***4.2.7 Method: `svc_api_get_alias_conf`**

Retrieve the alias configuration string in JSON format.

Returns:

/ *Type*: `str` /
The alias configuration string in JSON format.

4.2.8 Method: `is_specific_request`

Check if the request is a specific request.

Arguments:

- `request`
/ *Condition*: required / *Type*: object /
The request object to be checked.

Returns:

/ *Type*: bool /
True if the request is a specific request, otherwise False.

4.2.9 Method: `on_specific_request`

Handle the event when a specific request is received.

Arguments:

- `ch`
/ *Condition*: required / *Type*: `pika.channel.Channel` /
The channel object from the pika library.
- `method`
/ *Condition*: required / *Type*: `pika.spec.Basic.Deliver` /
The method object containing delivery information from the pika library.
- `props`
/ *Condition*: required / *Type*: `pika.spec.BasicProperties` /
The properties of the message from the pika library.
- `body`
/ *Condition*: required / *Type*: dict /
The body of the message as a dictionary.

Returns:

(no returns)

Chapter 5

Appendix

About this package:

Table 5.1: Package setup

Setup parameter	Value
Name	MicroserviceBase
Version	0.1.1
Date	02.02.2023
Description	TODO
Package URL	python-microservice-base
Author	TODO
Email	TODO
Language	Programming Language :: Python :: 3
License	License :: OSI Approved :: Apache Software License
OS	Operating System :: OS Independent
Python required	>=3.0
Development status	Development Status :: 4 - Beta
Intended audience	Intended Audience :: Developers
Topic	Topic :: Software Development

Chapter 6

History

0.1.0	02/2023
<i>Initial version</i>	

MicroserviceBase.pdf

Created at 11.03.2026 - 17:03:49

by GenPackageDoc v. 0.42.0 / 12.01.2026

7.2 MicroserviceClewareSwitch

MicroserviceClewareSwitch

v. 0.1.2

TODO

02.02.2023

Contents

1	Introduction	1
2	Description	2
2.1	ToDo	2
2.1.1	ToDo ToDo	2
3	ClewareAccessHelper.py	3
3.1	Method: get_config	3
3.2	Method: set_config	3
3.3	Method: init_cleware	4
3.4	Method: open_cleware	4
3.5	Method: close_cleware	4
3.6	Method: get_handle	4
3.7	Method: set_value	4
3.8	Method: get_value	4
3.9	Method: set_switch	4
3.10	Method: set_switch_by_port_name	4
3.11	Method: get_switch	4
3.12	Method: get_version	4
3.13	Method: get_usb_type	4
3.14	Method: get_serial_number	4
3.15	Method: valid_ser_number	4
3.16	Method: get_hw_version	4
3.17	Method: is_ampel	4
3.18	Method: iox	4
3.19	Method: get_all_devices_state	4
4	ClewareAccessHelperAbs.py	5
4.1	Method: open_cleware	5
4.2	Method: close_cleware	5
4.3	Method: set_value	5
4.4	Method: get_value	5
4.5	Method: set_switch	5
4.6	Method: get_switch	5
4.7	Method: get_handle	5
4.8	Method: get_version	5
4.9	Method: get_usb_type	5
4.10	Method: get_usb_type	5

CONTENTS

CONTENTS

4.11	Method: <code>get_serial_number</code>	5
4.12	Method: <code>valid_ser_number</code>	5
4.13	Method: <code>get_hw_version</code>	5
4.14	Method: <code>is_ampel</code>	5
4.15	Method: <code>iox</code>	5
5	ClewareAccessHelperLinux.py	6
5.1	Method: <code>init_cleware</code>	7
5.2	Method: <code>open_cleware</code>	7
5.3	Method: <code>close_cleware</code>	7
5.4	Method: <code>get_handle</code>	7
5.5	Method: <code>set_value</code>	7
5.6	Method: <code>get_value</code>	7
5.7	Method: <code>set_switch</code>	7
5.8	Method: <code>get_switch</code>	7
5.9	Method: <code>get_version</code>	7
5.10	Method: <code>get_usb_type</code>	7
5.11	Method: <code>get_serial_number</code>	7
5.12	Method: <code>valid_ser_number</code>	7
5.13	Method: <code>get_hw_version</code>	7
5.14	Method: <code>is_ampel</code>	7
5.15	Method: <code>iox</code>	7
5.16	Method: <code>get_all_sw_state</code>	7
6	ClewareAccessHelperWindows.py	8
6.1	Method: <code>init_cleware</code>	9
6.2	Method: <code>open_cleware</code>	9
6.3	Method: <code>close_cleware</code>	9
6.4	Method: <code>get_handle</code>	9
6.5	Method: <code>set_value</code>	9
6.6	Method: <code>get_value</code>	9
6.7	Method: <code>set_switch</code>	9
6.8	Method: <code>get_switch</code>	9
6.9	Method: <code>get_version</code>	9
6.10	Method: <code>get_usb_type</code>	9
6.11	Method: <code>get_serial_number</code>	9
6.12	Method: <code>valid_ser_number</code>	9
6.13	Method: <code>get_hw_version</code>	9
6.14	Method: <code>is_ampel</code>	9
6.15	Method: <code>iox</code>	9
7	ServiceBase.py	10
7.1	Class: <code>ResultType</code>	10
7.2	Class: <code>ResponseMessage</code>	10
7.2.1	Method: <code>get_json</code>	10
7.2.2	Method: <code>get_data</code>	10
7.3	Class: <code>ServiceBase</code>	10

CONTENTS

CONTENTS

7.3.1	Method: <code>parse_arguments</code>	10
7.3.2	Method: <code>parse_spec_arguments</code>	11
7.3.3	Method: <code>close</code>	11
7.3.4	Method: <code>create_request_data</code>	11
7.3.5	Method: <code>connect_broker</code>	11
7.3.6	Method: <code>serve</code>	11
7.3.7	Method: <code>register_service</code>	12
7.3.8	Method: <code>unregister_service</code>	12
7.3.9	Method: <code>request_service</code>	12
7.3.10	Method: <code>get_svc_api_methods_dict</code>	12
7.3.11	Method: <code>get_svc_api_methods_info_dict</code>	12
7.3.12	Method: <code>parse_docstring</code>	13
7.3.13	Method: <code>svc_api_get_version</code>	13
7.3.14	Method: <code>svc_api_get_gui_files</code>	13
7.3.15	Method: <code>is_specific_request</code>	13
7.3.16	Method: <code>on_specific_request</code>	13
7.3.17	Method: <code>on_request</code>	14
7.4	Docutils System Messages	14
8	ServiceCleware.py	15
8.1	Function: <code>signal_handler</code>	15
8.2	Class: <code>ServiceCleware</code>	15
8.2.1	Method: <code>svc_api_get_all_devices_state</code>	15
8.2.2	Method: <code>svc_api_set_switch</code>	15
8.2.3	Method: <code>notify_updates</code>	16
9	ServiceLogger.py	17
9.1	Method: <code>get_config_file</code>	17
9.2	Method: <code>log</code>	17
9.3	Method: <code>log_info</code>	17
9.4	Method: <code>log_debug</code>	17
9.5	Method: <code>log_warning</code>	17
9.6	Method: <code>log_error</code>	17
9.7	Method: <code>log_critical</code>	17
10	Utils.py	18
10.1	Class: <code>Singleton</code>	18
10.2	Class: <code>Utils</code>	18
10.2.1	Method: <code>get_all_sub_classes</code>	18
10.2.2	Method: <code>caller_name</code>	19
10.2.3	Method: <code>load_library</code>	19
10.2.4	Method: <code>is_ascii_or_unicode</code>	19
10.2.5	Method: <code>get_registry_parameters</code>	20
10.2.6	Method: <code>create_registry_parameters</code>	20
10.3	Class: <code>Job</code>	20
10.3.1	Method: <code>stop</code>	20
10.3.2	Method: <code>run</code>	20

CONTENTS	CONTENTS
10.4 Class: ResultType	20
10.4.1 Method: get_data	21
10.4.2 Method: create_from_string	21
11 Appendix	22
12 History	23
D	

Chapter 1

Introduction

TODO: Add introduction of **MicroserviceClewareSwitch**.

The sources of **MicroserviceClewareSwitch** are available in [GitHub](#).

Informations about how to install the **MicroserviceClewareSwitch** can be found in the [README](#).

Chapter 2

Description

2.1 ToDo

Description

2.1.1 ToDo ToDo

Description

ToDo ToDo ToDo

Description

Chapter 3

ClewareAccessHelper.py

ClewareAccessHelper class acts as a proxy class in Proxy pattern and forward the request to the real helper depend on platform. microserviceclewareswitch-clewareaccesshelper-clewareaccesshelper-load-config -----

Load the user config port name for USB access. Returns:

System Message

ERROR/3 in <string>, line 10
Unexpected indentation. backrefs:

None

System Message

WARNING/2 in <string>, line 11
Block quote ends without a blank line; unexpected unindent. backrefs:

3.1 Method: get_config

Get Cleware ports' config. Returns:

System Message

ERROR/3 in <string>, line 16
Unexpected indentation. backrefs:

Dictionary of ports' setting.

System Message

WARNING/2 in <string>, line 17
Block quote ends without a blank line; unexpected unindent. backrefs:

3.2 Method: set_config

Save Cleware ports' config to file. Args:

System Message

ERROR/3 in <string>, line 22
Unexpected indentation. backrefs:

config_json: data to be saved.

Returns: res: saved result.

System Message

WARNING/2 in <string>, line 26
Definition list ends without a blank line; unexpected unindent. backrefs:

- 3.3 Method: `init_cleware`
- 3.4 Method: `open_cleware`
- 3.5 Method: `close_cleware`
- 3.6 Method: `get_handle`
- 3.7 Method: `set_value`
- 3.8 Method: `get_value`
- 3.9 Method: `set_switch`
- 3.10 Method: `set_switch_by_port_name`
- 3.11 Method: `get_switch`
- 3.12 Method: `get_version`
- 3.13 Method: `get_usb_type`
- 3.14 Method: `get_serial_number`
- 3.15 Method: `valid_ser_number`
- 3.16 Method: `get_hw_version`
- 3.17 Method: `is_ampel`
- 3.18 Method: `iox`
- 3.19 Method: `get_all_devices_state`

Chapter 4

ClewareAccessHelperAbs.py

ClewareAccessHelperAbs acts as a abstract class for the real Helper in Proxy Pattern. microserviceclewareswitch-clewareaccesshelperabs-clewareaccesshelperabs-init-cleware -----

4.1 Method: open_cleware

4.2 Method: close_cleware

4.3 Method: set_value

4.4 Method: get_value

4.5 Method: set_switch

4.6 Method: get_switch

4.7 Method: get_handle

4.8 Method: get_version

4.9 Method: get_usb_type

4.10 Method: get_usb_type

4.11 Method: get_serial_number

4.12 Method: valid_ser_number

4.13 Method: get_hw_version

4.14 Method: is_ampel

4.15 Method: iox

Chapter 5

ClewareAccessHelperLinux.py

ClewareAccessHelperLinux acts as the real object on Linux platform in Proxy Pattern

- This is the wrapper class for the functions provided by C/C++ source for Linux.
- The C/C++ source for Linux is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN
- The C/C++ source for Linux is modified by Cuong Nguyen (RBVH/ENG22) for support python to load dynamic library on Linux.

System Message

WARNING/2 in <string>, line 8

Definition list ends without a blank line; unexpected unindent. backrefs:

- 5.1 Method: `init_cleware`
- 5.2 Method: `open_cleware`
- 5.3 Method: `close_cleware`
- 5.4 Method: `get_handle`
- 5.5 Method: `set_value`
- 5.6 Method: `get_value`
- 5.7 Method: `set_switch`
- 5.8 Method: `get_switch`
- 5.9 Method: `get_version`
- 5.10 Method: `get_usb_type`
- 5.11 Method: `get_serial_number`
- 5.12 Method: `valid_ser_number`
- 5.13 Method: `get_hw_version`
- 5.14 Method: `is_ampel`
- 5.15 Method: `iox`
- 5.16 Method: `get_all_sw_state`

Chapter 6

ClewareAccessHelperWindows.py

ClewareAccessHelperWindows acts as the real object on Windows platform in Proxy Pattern

- This is the wrapper class for the functions provided by USBaccess.dll.
- The USBaccess.dll is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN

System Message

WARNING/2 in <string>, line 7

Definition list ends without a blank line; unexpected unindent. backrefs:

- 6.1 Method: `init_cleware`
- 6.2 Method: `open_cleware`
- 6.3 Method: `close_cleware`
- 6.4 Method: `get_handle`
- 6.5 Method: `set_value`
- 6.6 Method: `get_value`
- 6.7 Method: `set_switch`
- 6.8 Method: `get_switch`
- 6.9 Method: `get_version`
- 6.10 Method: `get_usb_type`
- 6.11 Method: `get_serial_number`
- 6.12 Method: `valid_ser_number`
- 6.13 Method: `get_hw_version`
- 6.14 Method: `is_ampel`
- 6.15 Method: `iox`

Chapter 7

ServiceBase.py

7.1 Class: ResultType

Result Types.

7.2 Class: ResponseMessage

Response message class

7.2.1 Method: get_json

Convert the response message to JSON format.

Returns:

/ Type: str /

The response message in JSON format.

7.2.2 Method: get_data

Retrieve the result data as a string.

Returns:

/ Type: str /

The result data as a string.

7.3 Class: ServiceBase

7.3.1 Method: parse_arguments

Parse basic service arguments from the command line.

Arguments:

- cmd_args

/ Condition: required / Type: list /

Command-line arguments to be parsed.

Returns:

/ Type: dict /

A dictionary containing the parsed arguments.

7.3.2 Method: parse_spec_arguments

Parse specific arguments for each customized service.

This method should be overridden by subclasses to handle service-specific arguments.

Arguments:

- `cmd_args`
/ *Condition*: required / *Type*: list /
Command-line arguments to be parsed.

Returns:

/ *Type*: dict /
A dictionary containing the parsed specific arguments.

7.3.3 Method: close

Close the service connection.

Returns:

(no returns)

7.3.4 Method: create_request_data

Create request data for a given method name and arguments.

Arguments:

- `method_name`
/ *Condition*: required / *Type*: str /
The name of the method for which the request is being created.
- `args`
/ *Condition*: required / *Type*: list /
The list of arguments for the method.

Returns:

/ *Type*: dict /
A dictionary containing the method name and arguments.

7.3.5 Method: connect_broker

Establish a connection to the broker.

Arguments:

- `**kwargs`
/ *Condition*: optional / *Type*: dict /
Additional keyword arguments for broker connection parameters.

Returns:

(no returns)

7.3.6 Method: serve

Call to start service serving.

Returns:

(no returns)

7.3.7 Method: register_service

Register a service to the ServiceRegistry.

Returns:

(no returns)

7.3.8 Method: unregister_service

Unregister a service from the ServiceRegistry.

Returns:

(no returns)

7.3.9 Method: request_service

Send a service request to a specific exchange with a given routing key.

Arguments:

- request_data
/ Condition: required / Type: dict /
The data for the service request.
- exchange_name
/ Condition: required / Type: str /
The name of the exchange to send the request to.
- routing_key
/ Condition: required / Type: str /
The routing key for the request.

Returns:

(no returns)

7.3.10 Method: get_svc_api_methods_dict

Retrieve all service API methods provided by the service (methods starting with the prefix '**svc_api_**').

Returns:

/ Type: dict /
A dictionary containing the names and references of all service API methods.

7.3.11 Method: get_svc_api_methods_info_dict

Retrieve information for all service APIs from the docstrings of the methods.

Arguments:

- methods_dict
/ Condition: required / Type: dict /
A dictionary containing the names and references of all service API methods.

Returns:

/ Type: dict /
A dictionary containing the information of all service APIs extracted from their docstrings.

7.3.12 Method: parse_docstring

Parse function's docstring to get arguments and return information.

Arguments:

- `docstring`
/ *Condition*: required / *Type*: str /
Function's docstring.

Returns:

/ *Type*: str /
Payloads of the waiting signal if received.

7.3.13 Method: svc_api_get_version

Get the service version.

Returns:

/ *Type*: str /
Version of the service.

7.3.14 Method: svc_api_get_gui_files**7.3.15 Method: is_specific_request**

Check if the request is a specific request.

Arguments:

- `request`
/ *Condition*: required / *Type*: object /
The request object to be checked.

Returns:

/ *Type*: bool /
True if the request is a specific request, otherwise False.

7.3.16 Method: on_specific_request

Handle the event when a specific request is received.

Arguments:

- `ch`
/ *Condition*: required / *Type*: pika.channel.Channel /
The channel object from the pika library.
- `method`
/ *Condition*: required / *Type*: pika.spec.Basic.Deliver /
The method object containing delivery information from the pika library.
- `props`
/ *Condition*: required / *Type*: pika.spec.BasicProperties /
The properties of the message from the pika library.

- `body`
/ *Condition*: required / *Type*: dict /
The body of the message as a dictionary.

Returns:

(no returns)

7.3.17 Method: `on_request`

Handle an incoming request.

Arguments:

- `ch`
/ *Condition*: required / *Type*: `pika.channel.Channel` /
The channel object from the pika library.
- `method`
/ *Condition*: required / *Type*: `pika.spec.Basic.Deliver` /
The method object containing delivery information from the pika library.
- `props`
/ *Condition*: required / *Type*: `pika.spec.BasicProperties` /
The properties of the message from the pika library.
- `body`
/ *Condition*: required / *Type*: bytes /
The body of the message as bytes.

Returns:

(no returns)

7.4 Docutils System Messages**System Message**

ERROR/3 in <string>, line 186

Unknown target name: "svc_api".

Chapter 8

ServiceCleware.py

8.1 Function: `signal_handler`

Handle signals from the operating system.

Arguments:

- `sig`
/ *Condition*: required / *Type*: int /
The signal number received from the OS.
- `frame`
/ *Condition*: required / *Type*: frame object /
The current stack frame.
- `obj`
/ *Condition*: required / *Type*: object /
The object that is handling the signal.

Returns:

(no returns)

8.2 Class: `ServiceCleware`

Service for controlling Cleware devices.

This class extends `ServiceBase` to provide functionalities specific to managing and controlling Cleware devices.

8.2.1 Method: `svc_api_get_all_devices_state`

Retrieve the state of all Cleware devices.

Returns:

/ *Type*: dict /
A dictionary containing the states of all Cleware devices.

8.2.2 Method: `svc_api_set_switch`

Set state for a Cleware device's switch.

Arguments:

- `device_no`
/ *Condition*: required / *Type*: str /
Cleware device's number.
- `switch_id`
/ *Condition*: required / *Type*: str /
Switch number to turn on or off.
- `state`
/ *Condition*: required / *Type*: str /
State of swith to set (on/off).

Returns:

/ *Type*: int /
Return ret code, 1 for succeed, 0 for failure.

8.2.3 Method: notify_updates

Notify updates to the realtime update channel for Cleware devices.

Returns:

(no returns)

Chapter 9

ServiceLogger.py

Logger class. microserviceclewareswitch-servicelogger-servicelogger-set-logger -----

9.1 Method: get_config_file

Get a name of a caller in the format module.class.method Args:

System Message

ERROR/3 in <string>, line 13
Unexpected indentation. backrefs:

skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

System Message

WARNING/2 in <string>, line 18
Definition list ends without a blank line; unexpected unindent. backrefs:

9.2 Method: log

9.3 Method: log_info

9.4 Method: log_debug

9.5 Method: log_warning

9.6 Method: log_error

9.7 Method: log_critical

Chapter 10

Utils.py

10.1 Class: Singleton

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s)

System Message

ERROR/3 in <string>, line 9
Unexpected indentation. backrefs:

Used when class has too few public methods, so be sure it's really worth it.

This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

System Message

WARNING/2 in <string>, line 13
Block quote ends without a blank line; unexpected unindent. backrefs:

10.2 Class: Utils

Class to implement utilities for supporting development. microserviceclewareswitch-utils-utils-get-all-descendant-classes -----

Get all descendant classes of a class Args:

System Message

ERROR/3 in <string>, line 22
Unexpected indentation. backrefs:

cls: Input class for finding descendants.

Returns: Array of descendant classes.

System Message

WARNING/2 in <string>, line 26
Definition list ends without a blank line; unexpected unindent. backrefs:

10.2.1 Method: get_all_sub_classes

Get all children classes of a class Args:

System Message

ERROR/3 in <string>, line 31
Unexpected indentation. backrefs:

cls: Input class for finding children.

Returns: Array of children classes.

System Message

WARNING/2 in <string>, line 35
Definition list ends without a blank line; unexpected unindent. backrefs:

10.2.2 Method: caller_name

Get a name of a caller in the format module.class.method Args:

System Message

ERROR/3 in <string>, line 40
Unexpected indentation. backrefs:

skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

System Message

WARNING/2 in <string>, line 45
Definition list ends without a blank line; unexpected unindent. backrefs:

10.2.3 Method: load_library

Load native library depend on the calling convention. Args:

System Message

ERROR/3 in <string>, line 50
Unexpected indentation. backrefs:

path: library path. is_stdcall: determine if the library's calling convention is stdcall or cdecl.

Returns: Loaded library object.

System Message

WARNING/2 in <string>, line 55
Definition list ends without a blank line; unexpected unindent. backrefs:

10.2.4 Method: is_ascii_or_unicode

Check if the string is ascii or unicode Args:

System Message

ERROR/3 in <string>, line 60
Unexpected indentation. backrefs:

str_check: string for checking codecs: encoding type list

Returns: True : if checked string is ascii or unicode False : if checked string is not ascii or unicode

System Message

WARNING/2 in <string>, line 66
Definition list ends without a blank line; unexpected unindent. backrefs:

10.2.5 Method: get_registry_parameters

Get service's parameters which stored in registry. Args:

System Message

ERROR/3 in <string>, line 71
Unexpected indentation. backrefs:

service_name: Name of the service. key_name: Name of register key.

Returns: Value of service's parameters

System Message

WARNING/2 in <string>, line 76
Definition list ends without a blank line; unexpected unindent. backrefs:

10.2.6 Method: create_registry_parameters

Create service's parameters and store it to registry. Args:

System Message

ERROR/3 in <string>, line 81
Unexpected indentation. backrefs:

service_name: Name of the service. key_name: Name of register key. parameters: Parameters to be stored.

Returns: None

System Message

WARNING/2 in <string>, line 87
Definition list ends without a blank line; unexpected unindent. backrefs:

10.3 Class: Job

10.3.1 Method: stop

10.3.2 Method: run

10.4 Class: ResultType

Result Types. microserviceclewareswitch-utils-responsemessage =====

Response message class microserviceclewareswitch-utils-responsemessage-get-json -----

Convert response message to json Returns:

System Message

ERROR/3 in <string>, line 109
Unexpected indentation. backrefs:

Response message in json format

System Message

WARNING/2 in <string>, line 110

Block quote ends without a blank line; unexpected unindent. backrefs:

10.4.1 Method: get_data

Get string data result Returns:

System Message

ERROR/3 in <string>, line 115

Unexpected indentation. backrefs:

String result

System Message

WARNING/2 in <string>, line 116

Block quote ends without a blank line; unexpected unindent. backrefs:

10.4.2 Method: create_from_string

Chapter 11

Appendix

About this package:

Table 11.1: Package setup

Setup parameter	Value
Name	MicroserviceClewareSwitch
Version	0.1.2
Date	02.02.2023
Description	TODO
Package URL	python-microservice-cleware-switch
Author	TODO
Email	TODO
Language	Programming Language :: Python :: 3
License	License :: OSI Approved :: Apache Software License
OS	Operating System :: OS Independent
Python required	>=3.0
Development status	Development Status :: 4 - Beta
Intended audience	Intended Audience :: Developers
Topic	Topic :: Software Development

Chapter 12

History

0.1.0	02/2023
Initial version	

Chapter 8

Appendix

8.1 Installed Python Modules

This chapter contains a list of installed Python modules together with their version numbers.

Based on: Python 3.13.2 (main, Feb 5 2025, 19:16:35) [MSC v.1942 64 bit (AMD64)]

Identified 362 packages.

aenum : 3.1.16	cfv : 3.5.0	executing : 2.2.1
aio-pika : 9.5.8	chardet : 6.0.0.post1	fastjsonschema : 2.21.2
aiodns : 4.0.0	charset-normalizer : 3.4.4	filelock : 3.25.0
aiofiles : 25.1.0	click : 8.3.1	fonttools : 4.61.1
aiohappyeyeballs : 2.6.1	colorama : 0.4.6	fqdn : 1.5.1
aiohttp : 3.13.3	comm : 0.2.3	frozenset : 1.8.0
aiormq : 6.9.3	configparser : 7.2.0	func_timeout : 4.3.5
aiohttp : 1.4.0	contourpy : 1.3.3	GenPackageDoc : 0.42.0
alabaster : 1.0.0	coverage : 7.11.3	geographiclib : 2.1
altgraph : 0.17.5	cryptography : 46.0.3	geopy : 2.4.1
annotated-doc : 0.0.4	cycler : 0.12.1	h11 : 0.16.0
annotated-types : 0.7.0	Cython : 3.2.1	httpcore : 1.0.9
anyio : 4.12.1	debugpy : 1.8.20	httpserver : 1.1.0
appdirs : 1.4.4	decorator : 5.2.1	httpx : 0.28.1
Appium-Python-Client : 5.2.6	deepdiff : 8.6.1	humanfriendly : 10.0
argon2-cffi : 25.1.0	defusedxml : 0.7.1	identify : 2.6.17
argon2-cffi-bindings : 25.1.0	deprecation : 2.1.0	idna : 3.11
argparse-addons : 0.12.0	DevAtServ : UNKNOWN	ImageIO : 2.37.2
arrow : 1.4.0	@ : UNKNOWN	imagesize : 1.4.1
asciidoxy : 0.5.2	file:///C:/workplace/ROBFW/components/DevAtServ : 5.12.0	importlib-resources : 5.12.0
astroid : 4.0.4	documentation : UNKNOWN	imutils : 0.5.4
asttokens : 3.0.1	dill : 0.4.1	influxdb-client : 1.49.0
async-lru : 2.2.0	distlib : 0.4.0	iniconfig : 2.3.0
attrs : 25.4.0	docopt : 0.6.2	install-requires : 0.3.0
babel : 2.18.0	docutils : 0.22.4	invoke : 2.2.1
bcrypt : 5.0.0	doipclient : 1.1.7	ipykernel : 7.2.0
beautifulsoup4 : 4.14.3	dotdict : 0.1	ipython : 9.10.0
bincopy : 20.1.1	dotmap : 1.3.30	ipython_pygments_lexers : 1.1.1
bitstruct : 8.22.1	doxypy : 0.8.8.7	ipywidgets : 8.1.8
black : 25.11.0	EasyProcess : 1.1	isoduration : 20.11.0
bleach : 6.3.0	elementpath : 5.1.1	isort : 7.0.0
bokeh : 3.8.1	entrypoint2 : 1.1	jedi : 0.19.2
build : 1.4.0	et_xmlfile : 2.0.0	Jinja2 : 3.1.6
can-isotp : 2.0.7	EventBusClient : UNKNOWN	joblib : 1.5.3
cchardet : 2.2.0a2	@ : UNKNOWN	json5 : 0.13.0
certifi : 2025.11.12	file:///C:/workplace/ROBFW/components/DevAtServ : 3.0.0	jsonschema : 3.0.0
cffi : 2.0.0	rabbitmq-messagebus : UNKNOWN	JsonPreprocessor : 0.11.2

jenschema : 4.25.1	parso : 0.8.6	pyttss : 1.1
jenschema-specifications : 2025.9.1	path : 17.1.1	pytz : 2025.2
jupyter-events : 0.12.0	path.py : 12.5.0	pywin32 : 311
jupyter-lsp : 2.3.0	pathspecc : 1.0.4	pywin32-ctypes : 0.2.3
jupyter_client : 8.8.0	pdcc : 16.0.0	pywinpty : 3.0.3
jupyter_core : 5.9.1	pefile : 2023.2.7	PyYAML : 6.0.3
jupyter_server : 2.17.0	pickleDB : 1.3.2	pyzbar : 0.1.9
jupyter_server_terminals : 0.5.4	pika : 1.3.2	pyzmq : 27.1.0
jupyterlab : 4.4.10	pillow : 12.0.0	QLed : 1.3.1
jupyterlab_pygments : 0.3.0	platformdirs : 4.9.2	QtAwesome : 1.4.0
jupyterlab_server : 2.28.0	pluggy : 1.6.0	QtPy : 2.4.3
jupyterlab_widgets : 3.0.16	ply : 3.11	reactivex : 4.1.0
kitchen : 1.2.6	portalocker : 3.2.0	referencing : 0.37.0
kiwisolver : 1.4.9	portpicker : 1.6.0	regex : 2025.11.3
lark : 1.3.1	pre_commit : 4.4.0	requests : 2.32.5
lazy_loader : 0.4	pre_commit_hooks : 6.0.0	requests-kerberos : 0.15.0
lunr : 0.8.0	prometheus_client : 0.23.1	rfc3339-validator : 0.1.4
lxml : 6.0.2	prompt_toolkit : 3.0.52	rfc3986-validator : 0.1.1
Mako : 1.3.10	propcache : 0.4.1	rfc3987-syntax : 1.1.0
markdown-it-py : 4.0.0	protobuf : 6.33.0	rfdocsindexer : 1.3.0
markdown2 : 2.5.5	psutil : 7.1.3	rflw : 2.0
markdownify : 1.2.2	pure_eval : 0.2.3	rflwresourcelib : 2.0
MarkupSafe : 3.0.3	pycares : 5.0.1	rich : 14.3.3
matplotlib : 3.10.7	pycparser : 2.23	rich-click : 1.9.7
matplotlib-inline : 0.2.1	pydantic : 2.12.5	robotframework : 6.1
mccabe : 0.7.0	pydantic_core : 2.41.5	robotframework-appiumlibrary : 3.1
mdurl : 0.1.2	pydotdict : 3.3.11	robotframework-assertion-engine : 4.0.0
MicroserviceBase : 0.1.1	pyelftools : 0.32	robotframework-dbus : 0.1.3
MicroserviceCleaveSwitch : 0.1.2	pyfranca : 0.4.1	robotframework-dependencylibrary : 4.0.1
MicroserviceDebugboard : 0.1.0	Pygments : 2.19.2	robotframework-doctestlibrary : 0.28.2
mistune : 3.2.0	pyinstaller : 6.16.0	robotframework-doip : 0.1.6
mobly : 1.13	pyinstaller-hooks-contrib : 2026.1	robotframework-excellib : 2.0.1
MoblyLibrary : 1.1.0	pylibdmtx : 0.1.10	robotframework-extensions-collection : 0.10.1
msl-loadlib : 1.0.0	pylint : 4.0.2	robotframework-ftplib : 1.9
mss : 10.1.0	PyMuPDF : 1.27.1	robotframework-libtoc : 1.5.0
multidict : 6.7.1	PyNaCl : 1.6.1	robotframework-listenerlibrary : 1.0.3
mypy_extensions : 1.1.0	pyodbc : 5.3.0	robotframework-mqttlibrary : 0.7.1.post3
mysqlclient : 2.2.7	py pandoc : 1.16.2	robotframework-prometheus : 0.8.0
narwhals : 2.17.0	py pandoc_binary : 1.16.2	robotframework-pythonlibcore : 4.4.1
nbclient : 0.10.4	pyparsing : 3.2.5	robotframework-qconnect-base : 1.3.0
nbconvert : 7.17.0	pyproject_hooks : 1.2.0	robotframework-qconnect-dlt : 1.1.1
nbformat : 5.10.4	PyQt5 : 5.15.11	robotframework-qconnect-goepel : 0.5.0
nest-asyncio : 1.6.0	PyQt5-Qt5 : 5.15.2	robotframework-qconnect-winapp : 1.0.3
networkx : 3.6.1	PyQt5_sip : 12.18.0	robotframework-requests : 0.9.7
nodeenv : 1.10.0	pyreadline3 : 3.5.4	robotframework-robocop : 6.10.0
notebook : 7.4.7	pyrsistent : 0.20.0	robotframework-robotlog2db : 1.5.4
notebook_shim : 0.2.4	pyscreenshot : 3.1	robotframework-robotlog2rqm : 1.7.1
numpy : 2.3.4	pyserial : 3.5	robotframework-seriallibrary : 0.4.3
odxtools : 11.0.3	pyshark : 0.6	robotframework-sshlibrary : 3.8.0
opencv-python-headless : 4.13.0.92	PySocks : 1.7.1	robotframework-tidy : 4.18.0
openpyxl : 3.1.5	pyspnego : 0.12.0	robotframework-tmllog2robotlog : 1.2.0
ordereddict : 1.1	pytesseract : 0.3.13	RobotFramework_FDBus : 0.1.0
orderly-set : 5.5.0	pytest : 9.0.2	RobotFramework_TestsuitesManagement : UNKNOWN
orjson : 3.11.7	pytest-mock : 3.15.1	
outcome : 1.3.0.post0	PyTestLog2DB : 0.3.4	
packaging : 25.0	python-can : 4.6.1	
paho-mqtt : 2.1.0	python-dateutil : 2.9.0.post0	
pamqp : 3.3.0	python-discovery : 1.1.0	
pandas : 2.3.3	python-json-logger : 4.0.0	
pandocfilters : 1.5.1	python-robotframework : 2.5.0	
paramiko : 4.0.0	PythonExtensionsCollection : 0.16.0	
parsimonious : 0.11.0	pytokens : 0.4.1	

@ : UNKNOWN	Sphinx : 9.1.0	typer-slim : 0.24.0
file:///C:/workplace/ROBFW/components/robotframework	sphinxcontrib-applehelp : 2.0.0	typing-inspection : 0.4.2
testsuitesmanagement : UNKNOWN	sphinxcontrib-devhelp : 2.0.0	typing_extensions : 4.15.0
RobotFramework_UDS : 0.1.17	sphinxcontrib-htmlhelp : 2.1.0	tzdata : 2025.3
RobotFrameworkAIO : UNKNOWN	sphinxcontrib-jsmath : 1.0.1	udsoncan : 1.25.0
@ : UNKNOWN	sphinxcontrib-qthelp : 2.0.0	uri-template : 1.3.0
file:///C:/workplace/ROBFW/components/robotframework	sphinxcontrib-serializinghtml : 2.0.0	urllib3 : 2.5.0
documentation : UNKNOWN	ssplib : 0.5.0	virtualenv : 21.1.0
robotkernel : 1.6	stack-data : 0.6.3	Wand : 0.7.0
roman-numerals : 4.1.0	tabulate : 0.10.0	watchdog : 6.0.0
rpds-py : 0.30.0	termcolor : 3.3.0	Wave : 0.0.2
ruamel.yaml : 0.19.1	terminado : 0.18.1	wcwidth : 0.6.0
ruff : 0.14.4	TestResultDBAccess : 0.1.6	webcolors : 25.10.0
schedule : 1.2.2	threadpoolctl : 3.6.0	webencodings : 0.5.1
scikit-image : 0.26.0	tifffile : 2026.2.24	websocket-client : 1.9.0
scikit-learn : 1.7.2	tinycss2 : 1.4.0	wheel : 0.46.3
scipy : 1.16.3	tk : 0.1.0	widetsnbextension : 4.0.15
scp : 0.15.0	tkinterdnd2 : 0.4.3	windows-curses : 2.4.1
selenium : 4.39.0	toml : 0.10.2	wrapt : 1.17.2
Send2Trash : 2.1.0	tomli : 2.4.0	wsproto : 1.3.2
setuptools : 80.9.0	tomli_w : 1.2.0	xmlschema : 4.2.0
shellingham : 1.5.4	tomlkit : 0.14.0	xmldict : 1.0.2
six : 1.17.0	tornado : 6.5.4	xyzservices : 2025.11.0
sniffio : 1.3.1	traitlets : 5.14.3	yaml : 1.23.0
snowballstemmer : 3.0.1	trio : 0.33.0	
sortedcontainers : 2.4.0	trio-websocket : 0.12.2	
soupsieve : 2.8.3	typer : 0.24.1	

Additionally required Python packages can be installed in this way:

1. Windows:

```
"%RobotPythonPath%/python.exe" -m pip install --proxy <proxy address> <packagename>
```

2. Linux:

```
"${RobotPythonPath}/python3" -m pip install --proxy <proxy address> <packagename>
```

The full path and name of the Python interpreter is required in these command lines because the **RobotFramework AIO** installer does not modify the environment of the computer (except the setup of some environment variables).

The proxy address is an option and depends on the conditions under which your company grants the access to the internet.